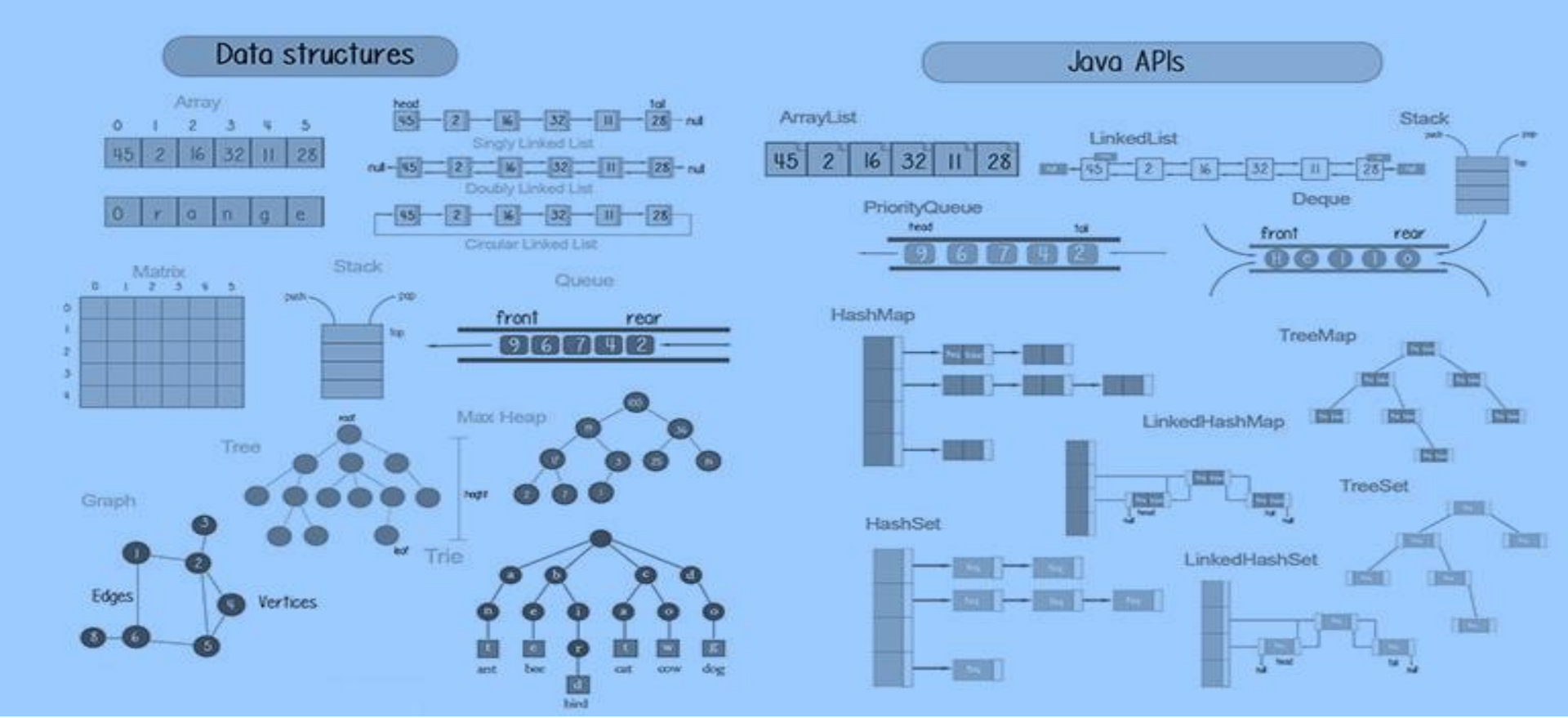


Java Programming

CPT111 – Lecture 10

Erick Purwanto

Xi'an Jiaotong-Liverpool University
西交利物浦大学

CPT111 Java Programming

Lecture 10

File I/O, List, Set, Map

Welcome to CPT111 – Lecture 10 !

- Find **a friend** to discuss today's lecture together in classroom
- Then, please join: Lecture 10 In-Class Quiz
- Who is this character in **Demon Slayer**?
  - Tanjiro
  - Zenitsu
  - Inosuke
  - Kyojuro
  - Nezuko

Welcome!

- Welcome to Lecture 10 !
- Last week, we have learned about OOP Principles and Exception
  - encapsulation, inheritance, and polymorphism
  - throwing and handling/catching exception
- In this lecture we are going to learn about
  - File I/O
    - learning Java File I/O mechanism
    - writing into and reading from a file
  - Lists
    - creating, accessing and manipulating a List as ArrayList and LinkedList
    - using List in method and in a class as an instance variable
  - Set and Map
    - Characteristics of and operations supported by Set and Map
    - using Set and Map to do problem solving

In-Class Quiz 1 – Review of Last Week

- Which concept in Java is primarily used to hide the internal details of an object and only expose necessary functionality?
  - Encapsulation
  - Inheritance
  - Polymorphism
  - Exception

Part 1: File I/O

- In the first part of lecture, we will learn about creating, writing to and reading from a file

File and File Class

- Data stored in a variable will be lost when a program terminates
- We need to save the data into **a file**, which is stored in a directory
  - the file is stored in a storage device
- **Absolute file name** – depends on the OS – for example in Windows:
  - C:\Users\erick.purwanto\Documents\IntelliJIDEAProjects\Lecture10Demo\data\text1.txt
- **Relative file name** – relative to the ***current working directory***
  - data\text1.txt
- In Java, we use a `File` object to represent a file
  - `File file = new File("C:\\Users\\erick.Purwanto\\Documents\\NetBeansProjects\\Lecture11Demo\\data\\text1.txt");`

File Object

- Since OS may vary, **do not** use absolute file name when creating `File` objects
- Use relative file name in Java (with forward slash) :
  `File file = new File("data/text1.txt");`
- Note that creating a File object **does not** actually create a file on the computer!

File Methods

- `File` class contains method to obtain the property of a file/directory

```
File file = new File("data/text1.txt");
file.exists();
file.isFile();
file.isDirectory();
file.isHidden();
file.length();
file.canRead();
file.canWrite();
file.getAbsolutePath();
```

in bytes

  - and to **rename** and **delete** a file/directory

    `delete(), renameTo(File), mkdir()`

however, to **read/write** into the file we need another class

Creating a new file using PrintWriter

- `PrintWriter` class is used to create a file and write data into a text file

```
File newFile = new File("data/text2.txt");
if (newFile.exists()) {
    System.out.println("File already exists!");
    System.exit(0);
}

try {
    PrintWriter output = new PrintWriter(newFile);
} catch (IOException ioe) {
    System.out.println(ioe.getMessage());
}
```

checking for existence first so that not overwritting

if so, print message and exit the program

otherwise, the new file will be created

Java forces us to handle this exception

Writing data into an existing file using PrintWriter

- `PrintWriter` class is used to create a file and write data into a text file

```
File file = new File("data/text1.txt");
try {
    PrintWriter output = new PrintWriter(file);
    output.println("Tanjiro");
    output.println("Zenitsu");
    output.println("Inosuke");
    output.close();
} catch (IOException ioe) {
    System.out.println(ioe.getMessage());
}
```

assuming file exists

then write similar to `System.out.println`

`close()` method must be used to write the data and close the file

- to create the file data/text1.txt you need to create the folder data first

In-Class Quiz 2

- What happens if we run the program again? 立即清空三行

```
File file = new File("data/text1.txt");
try {
    PrintWriter output = new PrintWriter(file);
    output.println("Tanjiro");
    output.println("Zenitsu");
    output.println("Inosuke");
    output.close();
} catch (IOException ioe) {
    System.out.println(ioe.getMessage());
}
```

  - 3 more lines added
  - no more lines added
  - there is an error message printed

Improve efficiency using BufferedWriter

- BufferedWriter stores the strings in memory before writing into the file

```
try {
    FileWriter file = new FileWriter("data/text1.txt");
    BufferedWriter buffer = new BufferedWriter(file);
    buffer.write("Tanjiro");
    buffer.newLine();
    buffer.write("Zenitsu");
    buffer.newLine();
    buffer.flush();
    buffer.write("Inosuke");
    buffer.newLine();
    buffer.close();
} catch (IOException ioe) {
    System.out.println(ioe.getMessage());
}
```

将缓冲区内容插入文件

use flush() method to force writing buffer content into file

otherwise write until buffer is full or closed

Reading text data using Scanner

- To read from a text file, create a Scanner for the file, read line-by-line:

```
File file = new File("data/text1.txt");
try {
    Scanner input = new Scanner(file);  真正开始读取文件
    while (input.hasNextLine()) {
        String name = input.nextLine();
        Swordsman swordsman = new Swordsman(name);  将文字变成对象
        System.out.println(swordsman);
    }
    input.close();
} catch (IOException ioe) {
    System.out.println(ioe.getMessage());
}
```

Reading csv data using Scanner

- To read from a csv (comma-separated values) file, create a Scanner for the file, extract the values by first splitting them:

```
File file = new File("data/demonslayerdata.csv");
try {
    Scanner input = new Scanner(file);
    while (input.hasNextLine()) {
        String line = input.nextLine();
        String[] values = line.split(",");
        String name = values[0];
        int numDemonsKilled = Integer.parseInt(values[1]);
        Swordsman swordsman = new Swordsman(name, numDemonsKilled);
        System.out.println(swordsman);
    }
    input.close();
} catch (IOException ioe) {
    System.out.println(ioe.getMessage());
}
```

extract data from a line, line-by-line

Improve efficiency using BufferedReader

- BufferedReader reads once from file into memory to accessed later

```
File file = new File("data/text1.txt");
try {
    FileReader fileReader = new FileReader(file);
    BufferedReader reader = new BufferedReader(fileReader);
    String line;
    while ((line = reader.readLine()) != null) {
        System.out.println(line);
    }
    reader.close();
} catch (IOException ioe) {
    System.out.println(ioe.getMessage());
}
```

access all lines line-by-line

Part 2: List, ArrayList and LinkedList 列表、数组列表和链表

- In the second part of lecture, we will learn about **lists**
  - how to create a List
  - how to add and access its data
  - how to insert and remove its data
  - how to use it for problem-solving

Array Deficiencies

- Say you want to create an array of Swordsman objects
- What if you don't know **how many objects** to store beforehand?
  - arrays need to declared with a size!
- What if you want to store **any numbers** of objects?
  - arrays cannot grow arbitrarily!

Java Collections and List

- Java provides a number of more powerful and flexible tools for managing collections of objects: the **Java Collections Framework**
- It includes List (this part) and Set, Map (next part of this lecture)
  - *both would be very useful for your CW3*

- A List contains an ***ordered*** collection of **zero or more** objects, where the ***same*** object might appear **multiple times**
  - we can add and remove items to and from the List, which will grow and shrink to accommodate its contents

列表（List）是一个有序集合，包含零个或多个对象，且同一对象可重复出现。○ 我们可以向列表中添加或移除项目，列表会根据内容动态扩展或收缩。

Creating Lists (1)

- Java helps us distinguish between the *specification* of a type – *what does it do?* – and the *implementation* – *what is the code?*
- List, Set and Map are all *interfaces*: they define **how** these respective types work, but they **don't** provide implementation code
  - For List, the implementations are (1) ArrayList or (2) LinkedList
- One advantage is that we, the users of these types, get to choose different implementations in different situations
- To create Lists, specify the object/generic type inside diamond <> operator:
  - `List<String> list1 = new ArrayList<String>();`
  - `List<String> list2 = new LinkedList<String>();`

接口interface　　实现类

Creating Lists (2)

- If the generic type parameters are the **same** on the left and right, Java can infer what's going on and save us some typing:
  - `List<String> list1 = new ArrayList<>();` 访问速度快get
  - `List<String> list2 = new LinkedList<>();` 插入速度快add

    本来<String>

    empty here. let's use this!

- `ArrayList` and `LinkedList` are two implementations of List
  - both provide all the operations of List, and those operations must work as described in the documentation for List
  - `list1` and `list2` will behave the same way, i.e. if we swap which one used `ArrayList` vs `LinkedList`, our code will **not** break

In-Class Quiz 3

- What do you think the resulting list would be after these list operations?

```
List<String> list = new ArrayList<>();
list.add("A");
list.add("B");
list.add("C");
list.add(1, "X");
list.set(2, "Y");
System.out.println(list);
```

  - [A X Y C]
  - [A B X Y C]
  - [A X B Y C]
  - [A X B C Y]

List Operations

- To declare a List variable, for example of integers, with an ArrayList:
  - `List<Integer> list = new ArrayList<>();`
- Some common operations:
  - adding `list.add(5)`
  - inserting `list.add(1, 3)`
  - assigning `list.set(2, 7)`
  - retrieving `list.get(2)`
  - length `list.size()`
  - remove `list.remove(1)`
  - remove all `list.clear()`

在1的位置插入3　插在原本的位置1的前面

wrapper class for primitives

adding to the back of list

replace the value at that index

- Create a list from an array

```
List<Integer> list = Arrays.asList(10, 20, 30)
```

List and ArrayList

```
List<Integer> list = new ArrayList<Integer>();
```

不能通过new来直接创造

- `List` is an **interface**, a type that can't be constructed directly with `new`, but that instead *specifies the operations* that a List must provide
  - `ArrayList` is a **class**, a concrete type that provides *implementations of those operations*

    提供了实现方法
  - `ArrayList` isn't the only implementation of the `List` type, though it's the most commonly used one
    - `LinkedList` is another implementation

Object type / generic type parameter

```
List<Integer> list = new ArrayList<Integer>();
```

- We wrote `List<Integer>` instead of `List<int>`
  Lists only know how to deal with *object types*, not *primitive types*
  - In Java, each of the primitive types (lowercase, abbreviated) has an equivalent object type (capitalized, fully spelled out)
  - Java requires us to use these object type equivalents when we **parameterize** a type with diamond operator **< >** angle brackets

ArrayList vs LinkedList

- Two possible implementations of a List:

	`ArrayList`	`LinkedList`
Actual data structure	Resizing Array	Doubly Linked List
Storing and accessing	Fast	Slow
Inserting and deleting	Slow	Fast

you will actually create both yourself from scratch in the Data Structure course

Iterate through List

- You can use the **enhanced for loops** to iterate through a list
  - for example:

```
for (int x : list) {
    System.out.print(x + " ");
}
```

- Demo examples:
  - Write a function `findMaxInt` to find the maximum element in an input `List` of integers using the enhanced for loop
  - Write a function `readFile` that takes a file name and returns a `List` of `Swordsman` objects generated from the file
  - Write a `DemonSlayerCorps` class that uses a `List` to store a troops of `Swordsman` objects

FindMaxInt

- Write a function `findMaxInt` to find the maximum element in an input `List` of integers using the enhanced for loop

```
public static int findMaxInt(List<Integer> list) {
    int max = list.get(0);
    for (int num : list) {
        if (num > max) {
            max = num;
        }
    }
    return max;
}
```

Reading from a file and returning List of Swordsman

- Write a function `readFile()` that takes a file name and returns a `List` of `Swordsman` objects generated from the file!
  - the file contains names of the `Swordsman`

```
public static List<Swordsman> readFile(String fileName) {
    File file = new File(fileName);
    List<Swordsman> list = new ArrayList<>();
    try {
        Scanner input = new Scanner(file);
        while (input.hasNextLine()) {
            String name = input.nextLine();
            Swordsman swordsman = new Swordsman(name);
            list.add(swordsman);
        }
        input.close();
    } catch (IOException ioe) {
        System.out.println(ioe.getMessage());
    }
    return list;
}
```

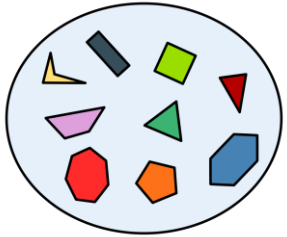
DemonSlayerCorps Class

- <u>Exercise</u>:
  Write a `DemonSlayerCorps` class that uses a `List` to store a troops of `Swordsman` objects!
  - complete it with constructors, and
  - instance methods such as `heal()` that heals the whole troops

Part 3: Set and Map

- In the third part of lecture, we will learn about Set and Map
- Java provides a number of powerful and flexible tools for managing collections of objects: the **Java Collections Framework**
- It includes List, Set and Map
  - *they would be very useful for your CW3*

Set



- A `Set` stores a unordered group of zero or more unique objects
  - no sense of order: either an item is in the set, or not
  - no duplicates: an object cannot appear in a `Set` multiple times

- Two popular `Set` implementations in Java:
  - `HashSet` – implemented using a hash table
  - `TreeSet` – implemented using a self-balancing tree data structure

- Other implementations: `LinkedHashSet` and `EnumSet`
- Will learn more in Data Structure and Advanced OOP courses

Set Operations

- To declare a `Set`, for example of integers, with a `HashSet`:
  - `Set<Integer> set = new HashSet<>();`
- Some common operations:
  - adding `set.add(5)`
  - checking membership `set.contains(5)`
  - remove `set.remove(5)`
  - union `set.addAll(c)`
  - intersection `set.retainAll(c)`
  - difference `set.removeAll(c)`
  - contain/subset `set.containsAll(c)`
  - size `set.size()`
  - clear contents `set.clear()`

c: other collection, e.g. other set

adding (no concepts of order)

present or not

set ∪ c

set ∩ c

set \ c

check whether set ⊇ c

Iterate through Set

- You can use the **enhanced for loops/for each loop** to iterate through a `Set`

```
public static void main(String[] args) {

    Set<String> set = new HashSet<>();
    set.add("B");
    set.add("C");
    set.add("A");

    System.out.println(set);

    for (String string : set) {
        System.out.println(string);
    }
}
```

In-Class Quiz 4

- What will be printed when we check membership of "D" ?

```
public static void main(String[] args) {

    Set<String> set = new HashSet<>();
    set.add("B");
    set.add("C");
    set.add("A");

    System.out.println(set.contains("D"));

}
```

  - null
  - false
  - there is an exception message printed

Set Operations – Example 1

- Creating set from list or other set, union, test containment, intersection:

```
public static void main(String[] args) {

    Set<Integer> set1 = new HashSet<>();
    set1.addAll(Arrays.asList(new Integer[] {1, 3}));
    Set<Integer> set2 = new HashSet<>();
    set2.addAll(Arrays.asList(new Integer[] {2, 5, 4}));

    set1.addAll(set2);
    System.out.println("set1 = set1 ∪ set2 = " + set1);

    System.out.println(set1.containsAll(set2));

    Set<Integer> intersectionSet = new HashSet<>(set1);
    Set<Integer> set3 = new HashSet<Integer>();
    set3.addAll(Arrays.asList(new Integer[] {1, 5, 6, 7}));
    intersectionSet.retainAll(set3);
    System.out.println("set1 ∩ set3 = " + intersectionSet);
}
```

creating set from list (that's created from array)

creating set from another set

Set Operations – Example 2

- Set difference and remove an element:

```
public static void main(String[] args) {

    Set<Integer> set4 = new HashSet<>();
    set4.addAll(Arrays.asList(new Integer[] {1, 2, 3, 4, 5}));
    Set<Integer> set5 = new HashSet<>();
    set5.addAll(Arrays.asList(new Integer[] {2, 5}));

    Set<Integer> setDifference = new HashSet<>(set4);
    setDifference.removeAll(set5);
    System.out.println("set4 \\ set5 = " + setDifference);

    setDifference.remove(1);
    System.out.println(setDifference);
}
```

Map

- A **map** stores key/value pairs, where each **key** has an associated **value**
  - Given a particular key, the map can look up the associated value *very quickly* !

- A map is also known as a **dictionary** or a **symbol table**

- Implementation of maps are `TreeMap`, `HashMap` and `LinkedHashMap`

Map Operations

- Operations in maps include:
  - `map.put(key, val)` adding the mapping key → val
  - `map.get(key)` get the value for a key
  - `map.containsKey(key)` test whether the map has the key
  - `map.containsValue(value)` test whether the map has the value
  - `map.remove(key)` delete a mapping
  - `map.keySet()` returns a set containing all keys

In-Class Quiz 5

- What will be printed when we look up "Kyojuro" ?

```
Swordsman tanjiro = new Swordsman("Tanjiro", 5);
Swordsman zenitsu = new Swordsman("Zenitsu", 2);
Swordsman inosuke = new Swordsman("Inosuke", 10);

Map<String, Swordsman> troops = new HashMap<>();

troops.put("Tanjiro", tanjiro);
troops.put("Zenitsu", zenitsu);
troops.put("Inosuke", inosuke);

Swordsman swordsman = troops.get("Kyojuro");
System.out.println(swordsman);
```

  - null
  - false
  - empty string / nothing is printed
  - there is an exception message printed

Map Operations Examples and Iteration

```
Map<String, Swordsman> troops = new HashMap<>();

troops.put("Tanjiro", tanjiro);
troops.put("Zenitsu", zenitsu);
troops.put("Inosuke", inosuke);

Swordsman swordsman;
swordsman = troops.get("Kyojuro");

if (troops.containsKey("Tanjiro")) {
    swordsman = troops.get("Tanjiro");
    // do something with swordsman
}

for (String key : troops.keySet()) {
    System.out.println(key + ": " + troops.get(key));
}
```

create an empty map

we can iterate over the keys of a map

Map Example : NumWords

- Suppose we have a list of strings of multiple duplicate words, such as ["a", "b", "a", "c", "b", "word", "a", "word"]
- We want to find out which words appear in the list and how many times each one appears
- Here is an algorithm NumWords using Map:
  - Create a Map<String, Integer>
  - Loop through all the strings in the list
  - Use each string as a key into the map
  - Use the Integer value to store the number of times that string has been seen
  - There are 2 cases:
    - The first time we see a word, it is not yet in the map, we add it
    - The later times we see the word, it is already in the map, we update it

NumWords Code

```java
import java.util.HashMap;
import java.util.Map;

public class NumWords {
    public Map<String, Integer> wordCount(List<String> list) {
        Map<String, Integer> map = new HashMap<>();
        int num;
        for (String word : list) {
            if (!map.containsKey(word)) {
                map.put(word, 1);
            }
            else {
                num = map.get(word);
                map.put(word, num + 1);
            }
        }
        return map;
    }
}
```

the first time we've seen string `word`

we increment the number of `word`

In-Class Quiz 6 – Have you learned something today?

- Which Java collection preserves insertion order and allows duplicate elements?
  - List
  - Set
  - Map

Thank you for your attention !

- **Conclusion:** in this lecture, you have learned:
  - how to create, read and write files
  - how to create, read and write files
  - how to create, access and manipulate a List implementation with ArrayList and LinkedList
  - how to use Set and Map to solve problems

- Please continue to Lab 10 to complete Lab Tasks, and then solve
  - Exercise and CW1 of Week #10